

U7280249 COMP1130 Assignment 2 - Technical Report

NOTE: A pdf version of this document is located the same directory

Introduction

Running the command `cabal v2-run turtle` in the console will open the assignment. The user is prompted with a URL that takes them to a webpage titled 'Blank Canvas'. The program responds to the following keys:

Key:	Effect:
M	Letters 'COMP1100'
I	Letters 'HI'
T	Equilateral Triangle
P	Polygon (n Sides)
H	Hilbert Curve (Order n)
C	Sierpinski Curve (Order n)
=/ -	Increase/Decrease the value of n
Z	Change view of fractal

Every time the user changes the shape displayed, the value of n resets. The two different display modes of fractals are: bounded area; constant line length.

Task 1 – Polygon Drawing

The first task involved designing two functions that would produce a polygon.

The `triangle` function takes a side-length argument and produces an equilateral triangle in the form of a `[TurtleCommand]`. The function is limited as the triangle can only be rotated at a higher-level.

The `polygon` function takes a side number and length, producing a regular polygon as a `[TurtleCommand]`. It is also assumed that it will be rotated at a higher level.

Implementing this function involved using recursion to produce n repeated actions of turning and moving. The angle it turned by was equal to the exterior angle of a polygon: $2 * \pi / n$. This also required a helper function that appended new move sets to the end of a list.

Testing involved pressing the ‘T’ or ‘P’ keys and incrementing the value of n to check the first few polygons. This was done after task 2 was completed. This visual check is adequate.

Task 2 – Turtle Visualisations

Task 2 involved being able to convert a series of turtle commands into a `CodeWorld` picture using the `runTurtle` function.

I made a `Pen` data type, which would store important information about the state of the turtle. Along with this, I made an initial state for this pen.

The accumulator function associated with `runTurtle` acts to repeatedly update the value of the pen and the final picture, while moving through each element in the provided set of commands. I found this difficult due to the non-Imperative nature of Haskell.

The helper function `affectPen` would change the state of the pen given a single turtle command. It is treated as top-level only for convenience in Task 4.

A major technical issue involved the improper development of the helper function `affectPen`. The problem was that I did not update the stored old position of the pen. I applied a white-box testing method to `affectPen` by analysing the edge cases, which is how I discovered the error.

Testing involved the pre-made `[TurtleCommand]`, as well as a simpler list used for debugging purposes.

Task 3.1 – L-Systems & The Hilbert Curve

In order to fully implement this part of the task, I had to modify the `Main.hs` file. I did not modify any of the pre-existing styling choices made.

The L-System data type contained a seed, and a production rule – which was a set of tuples with the first element being the alphabet token, and the list being what to replace it with.

E.g. (A, PBFNAFANFBP) means replace A with PBFNAFANFBP.

In order to make the code more modular and readable, I created a generic `TurtleAlphabet` data type. This was so I could re-use the same constructors for multiple L-Systems. I could not use the `+/` characters as constructors, as Haskell wouldn't recognise them as valid constructors.

The next step was to develop a function `interpretLSystem` that would iterate a given L-System, `n` times. This involved the use of a helper function `expand` which would perform a single iteration, and a second function `expandSingle` which would expand only one L-System token. This was done by checking each entry in the production rule for a match, then substituting it in.

Implementing the Hilbert curve involved using a `foldr` function in order to produce a `[TurtleCommand]`. A helper function was used to convert a alphabet token into its corresponding turtle command. Extra commands were appended at the front to make it functional.

To test this, I pressed the 'H' key and incremented/decremented to observe if it produced a valid set of Hilbert curves. It is assumed that the rotation of the curve is not an issue, nor is the response times for higher orders. Visually checking is adequate enough to confirm that the code is correct.

Task 3.2 – The Sierpinski Curve

This part of the task involved the development of my own L-System meant to represent an Sierpinski triangle. I chose this fractal as it is easily recognisable, and is self-similar (which is necessary for a simple L-System).

The L-System has the following tokens, the pointer initially is rotated by 60 degrees to the left:

Token	Product Rule	Turtle Command	Purpose
T	TFAT+FA-T-FA+	F+F+F+	A complete level-0 triangle, meant to iterate to form a level 1 triangle
A	FAA	--	Meant to double the amount of Fs
F	--	Move forward	
P (+)	--	Turn 120 degrees left	
N (-)	--	Turn 120 degrees right	

In order to develop this set of tokens, I considered the relationship between a curve of order n and of order $n + 1$:

$$[0] = F + F + F +$$
$$[1] = [0] F [0] + F - [0] - F +$$
$$[2] = [1] FF [1] + FF - [1] - FF +$$

This pointed out that the lower-level triangles should iterate to the next order up, and the amount of Fs should double each time. From this, implementing the L-System was similar to the Hilbert curve, with different meanings for tokens.

A limitation of this L-System is that the pointer draws over several already-traversed locations, which is inefficient for `CodeWorld`, but at higher orders, this becomes negligible in terms of processing time.

To test, I did the exact same thing as the Hilbert curve, instead using the 'S' key. Likewise this would demonstrate that it could produce the desired shape.

Task 4 – Testing

All functions were additionally white-box tested where possible, in addition to what was previously described. Only notable examples where I found errors were mentioned earlier.

In addition to this, I modified the `TurtleTest.hs` file to include five black-box tests.

The first two tests were meant to ensure that I produced the correct L-System iteration for an order 2 Hilbert/Sierpinski curve.

The third test checked if the code a series of L-System tokens to turtle commands, in which I used an order 1 Hilbert curve to check this.

The fourth test checked that a 3-sided polygon would equal a triangle, meant to affirm that both functions in Task 1 were correct.

The fifth test tested the `affectPen` function.

I assumed that all tests involving `Picture` outputs could be tested manually and visually by manipulating the canvas after running `cabal`.

Task 5 – Styling

I re-formatted the files `Turtle.hs` and `TurtleTest.hs` in accordance to the ANU style guide.

This involved several tasks: shortening all lines below 80 characters; including function descriptions; modularising code; and renaming identifiers where necessary.

Most time was spent modularising task 3, as there were many repeated sections of code involved in producing both fractal curves.

It is assumed that I did not need to worry about styling for `Main.hs`.

Task 6 – Technical Report

In writing this document, I intended to be more well-organised in my sentencing, and I tried to respond to feedback from last assessment.

Overall Reflection

The main conceptual/technical issues came in Task 2, with displaying images, and Task 3 with inventing a new L-System. However, these were solved by white-box testing and critical thinking respectively.

I have attempted to make the code readable by standardising the style and using helper functions where possible. I find that pattern matching helper functions were especially useful, notable examples were `affectPen` and `alphabetToCommand`.

When starting Task 3, I noticed that the Hilbert curve was acting very strange when using the ‘Z’ key, as well as it wasn’t positioning itself in the centre of the canvas. I modified this so that pressing the ‘Z’ key would switch between an unaffected Hilbert curve and a curve centred and contained within the co-ordinate plane.